Cloud Computing

Exercise - 8 Total points: 10

Let us extend our exercise 7 to learn more about Docker

1. Deploy Flask app container with a database container for storage

Deploy flask application with linking to PostgreSQL relational database container to store the data.

- Firstly run the Postgresql database container using following command and set the value of postgresql configuration variable `POSTGRES_PASSWORD` according to your convenience in `docker run -d --name postgres -e POSTGRES_USER=postgres -e POSTGRES_PASSWORD=<your_paasword> -e PGDATA=/var/lib/postgresql/data/pgdata -v /tmp:/var/lib/postgresql/data -p 5432:5432 -it postgres:14.1-alpine`
- Now let us clone the modified code to store messages in the postgresql container. You use the clone as `git clone` https://github.com/cloud-and-smart-labs/DDPC-2024-ex2.git `lab2app`.

  In this code, we used SQLAlchemy with Flask framework to interact with Postgresql.

- Change the current directory to lab2app and copy the Dockerfile of exercise 7 in this directory. Make relevant changes in the Dockerfile
- Modify the home.html to display your name and Exercise/task number
- Rebuild the image with name <flask_task2_lastname>
- Now run the container with following options:
  - detached mode (-dit)
  - Container image: `<flask_task2_lastname>`
  - Name of container: `<task2_your_lastname>`
  - port mapping : host(80), container(5000)
  - Set environment variable for linking the postgres database (-e): `DATABASE_URL=postgresql://postgres:<your_paasword>@<postgresContainer_IP>:5432/postgres`
  - The whole command for example should look like: `docker run -dit -p 80:5000 --name flask -e DATABASE_URL=postgresql://postgres:4y7sV96vA9wv46VR@172.17.67.119:5432/postgres flask_task2_srirama`
    - NB! make sure to change the password and the IP address in this command to match the information of your postgres container.
- By now, you should see the application running at http://VM_IP:80 and add few messages.
- NB!! Take the screenshot of running containers and output of web page (1 point)
- Stop and remove the postgresql container
- Again create postgresql container and still you can see the previous messages in the flask application. The volumes are important to persistent your data in the host even though the container is deleted or killed.

2. Working with docker compose

Docker Compose is a tool for defining and running multi-container Docker applications. With Compose, you use a YAML file to configure your application's services. Then, with a single command, you create and start all the services from your configuration. To learn more about all the features of Compose, see the list of features.
(https://docs.docker.com/compose/#features)

The compose has commands for managing the whole lifecycle of your application:

1. Start, stop, and rebuild services
2. View the status of running services
3. Stream the log output of running services

In this exercise, we are going to use docker-compose commands to start/stop/view status multiple services (flask application and Postgresql database service). We have two tasks to perform:

1. Task 1: Working with the docker-compose commands for deploying Message-Board application that stores the data in the directory.
2. Task 2: Working with the docker-compose commands for deploying Message-Board flask application that is linked to Postgresql.

Setup the docker-compose service using `sudo apt install docker-compose`

Task 2.1

Working with the docker-compose commands for deploying Message-Board application that stores the data in directory.

- Go to the directory of the *data.json* -version of the flask application (lab1app directory)
- Create a docker-compose file `nano docker-compose.yml` to define a new Docker service called *flask_app* with following options:
  - port mapping host(80), container(5000)
  - Volume (-v) `.:/lab1app`

```
version: "3.7"
services:
  flask_app:
    build: .
    ports:
      - "80:5000"
    volumes:
      - .:/lab1app
```

- Now, run the docker-compose `docker-compose up`, this will build the image and create the application container in non-detached mode. When you cancel the docker-compose task it automatically deletes the container.
- You can try to run in detached mode using `docker-compose up -d` and check the running container at http://VM_IP:80 and add a few messages. You can also check the contents of data.json in your host machine.
- Check the list of docker-compose services `docker-compose ps`
- Stop the services using command `docker-compose stop`
- Try **at least two other commands** from this list
- NB! Take the screenshot of the output of the commands that you tried. (2 points)

Task 2.2

Working with the docker-compose commands for deploying Message-Board flask application that linked to Postgresql.

This task is to be carried out by yourself with the knowledge of the previous task and reading docker-compose documentation.

- Create docker-compose.yml with 2 services (flask_app, postgresql).
  - As a starting point, you can use this "skeleton":

```
version: "3.7"

services:

  flask_app:


  postgresql:


volumes:

  db:
```

- For **flask_app**:
  - **Build path** should be the location of Dockerfile
  - You need to use tag `depends_on` to link the flask_app application to postgresql service (Example) (https://docs.docker.com/compose/how-tos/startup-order/)
  - Set **environment variables**, here DATABASE_URL=postgresql://postgres:<postgresql_password>@postgresql:5432/postgres (Example) (https://docs.docker.com/compose/how-tos/environment-variables/)
    - Make sure to change the PASSWORD to match the PostgreSQL password
  - Set port mapping to map host port 80 to container port 5000
- For **postgresql**:
  - Set **image** as `postgres:14.1-alpine`
  - Set **environment variables** `POSTGRES_USER=postgres` and `POSTGRES_PASSWORD=postgres`

    - We do not need to map ports, because the database container does not have to be accessible externally through the host machine ports.
    - **Assign a Volume** where the database files will be stored `db:/var/lib/postgresql/data`
- Define which **volumes should be created**:
    - Define the `db` volume as a separate entry at the end of the docker-compose.yml file (outside and after the "services:" entry)
        - `volumes:`
        - `  db:`
        - `   name: postgres-db-vol`
        -
- Create and run the services using `docker-compose up` (do NOT use -d).
    - As a result, you get 2 service containers running, plus 1 data volume, all from a single command!
- NB! Take the screenshot of `docker-compose up` command output (2 points)

3. Installation of Kubernetes on the system (single-node setup)

k3s is a lightweight Kubernetes distribution designed for edge computing, IoT, and small-scale Kubernetes deployments. It consolidates essential Kubernetes components into a single binary, reduces the resource consumption and simplifying deployment.

Some important terms:

1. *Pod: a group of one or more containers, which share the same network (they are the smallest units of workload in Kubernetes)*
2. *Control-Plane: the admin layer that runs the most important Kubernetes components.*
3. *Worker: a compute unit that doesn't run any control plane components.*
4. *Node: The physical / virtual machine which can run workloads.*
5. *CNI: Container Network Interface, which facilitates communication between container run-times and provides networking for the Kubernetes cluster (1 or more nodes).*
6. *Namespace: a logical partition used to organize and isolate resources within a cluster.*

The main node (single-node deployment of k3s) itself acts as control-plane (master) and the worker. kubectl is a command-line tool used to interact with the cluster.

- Download and install k3s (Downloads the k3s installation script from the official source, and executes the script)
  `curl -sfL https://get.k3s.io | sh -`
  What it does: It installs k3s as a systemd service, configures kubectl, sets up containerd (lightweight container runtime which is used instead of Docker) and uses Flannel for CNI (networking).
- To verify that k3s is running:
  `sudo systemctl status k3s`
  To verify: Active: active (running) confirms that k3s is running successfully.
- Verifying the version of k3s installed:
  `k3s --version`
- Setting up the kubeconfig for Kubernetes CLI: (This provides the information of cluster configuration to kubectl)
  `export KUBECONFIG=~/.kube/config`
  What it does: Kubeconfig is a configuration file that allows kubectl to communicate with the Kubernetes API server.

- Creating the .kube directory (if it doesn't exist already)
  `mkdir -p ~/.kube`
- Copy the k3s kubeconfig file to the default location
  `sudo cp /etc/rancher/k3s/k3s.yaml ~/.kube/config`
  What it does: k3s stores its kubeconfig file at: /etc/rancher/k3s/k3s.yaml by default. Hence, to ensure kubectl command has access to the cluster, we copy it to kubeconfig location.
- To change the ownership of the kubeconfig file:
  `sudo chown $(whoami):$(whoami) ~/.kube/config`
- To check if all the nodes in the cluster are up and running:
  `kubectl get nodes`

4. Working with Pods

Pods are the smallest, most basic deployable objects in Kubernetes. They can hold one or a group of containers, such as Docker containers which share the storage, network, and other resources.

- Let us create a pod with one container (container image: busybox)

  ```
  k3s kubectl run busybox --image=busybox --command -- /bin/sh -c
  "echo Hello Kubernetes! && while true; do sleep 3600; done"
  ```

  - Pod name is busybox
  - Pod with container image busybox (--image=busybox)
  - Run with shell command *echo Hello Kubernetes!* and sleep for 30 seconds in a while loop (/bin/sh -c "echo Hello Kubernetes! && while true; do sleep 3600; done")

- Check for the pod creation: `k3s kubectl get pods`
- Check the description of the pod

  ```
  k3s kubectl describe pod busybox
  ```

- Check for the details of pod, for example which node the pod is running

  ```
  k3s kubectl get pods -o wide
  ```

- To check the logs of the pod using *kubectl logs <podname>*. *Ex:* `k3s kubectl logs busybox`

  (You can see the output of the command that busybox shell is printed - Hello Kubernetes!)

- You can also exec the pod and open a command line inside the pod using `k3s kubectl exec -it busybox -- /bin/sh` and you can exit the pod using `exit` command
- You can delete the pod using

  ```
  kubectl delete pod busybox
  ```

```
kubectl delete pods --all
```

5. Kubernetes YAML File Setup

Deployment is used to manage the lifecycle of various applications in Kubernetes. The application's state is defined using Kubernetes Deployment YAML file, which allows Kubernetes to handle scaling, self-healing in case of failures, rolling updates, etc.

To create a Deployment, the resource must be defined as a .yaml file with essential configuration details like the number of replicas, container images, labels for the pods, etc.

YAML files can have two structures defined in the content.

I. *Maps: where the information is represented in the form of name:value (key, value) pairs.*
II. *Lists: A sequence of objects*

The files allows us to define map of maps, list of lists, map of lists and list of lists for various applications.

NB! Don't use TABS in the YAML files and ensure proper indentation of all the key:value pairs according to the hierarchy in which they are defined.

Some of the main components of the YAML file which are used in this exercise are:

Elements of YAML file	Description
---	Separator (used when multiple structures are defined in a single file)
- item	To define items in a list within the YAML file
apiVersion:	It specifies the version of the Kubernetes API that the resource adheres to (for compatibility between the YAML file and the cluster) • v1 for basic objects like Pod, service, etc. • apps/v1 for higher-level objects like Deployment, etc.
Kind	It defines the type of resource being created or modified. • Pod – runs container(s) • Service – exposes applications • Deployment – manages stateless applications • StatefulSet – manages stateful applications • PersistentVolume (PV) – represents storage • PersistentVolumeClaim (PVC): Requests storage
metadata	Contains information about the resource like name, labels, namespaces, and annotations. • name – name of the resource for unique identification in the namespace. • labels - grouping of resources based on key-value pairs. • annotations – additional information or metadata about the resource
Spec	Describes the desired state of the resource as either Deployment, Service, Pod, ConfigMap, etc.
replicas	Specifies the number of pod instances to run
selector	Maps the Service to Pods with matching labels.
template	Defines the pod specification for the deployment
containers	Defines the list of containers present in the pod. In this, the name, image to be used, environment variables and ports can be configured.
Type	It defines how the service is exposed. • ClusterIP – for internal access pod-to-Pod communication within the same cluster • NodePort – allows external access to the pods • LoadBalancer – creates an external LoadBalancer
externalTrafficPolicy	Controls how external traffic is handled • Local: Routes traffic only to the node where the pod is running. • Cluster: Balances traffic across all nodes.

More information about configuring .yaml files for Kubernetes Deployment can be found at:

I. *https://kubernetes.io/docs/concepts/overview/kubernetes-api/*
II. *https://kubernetes.io/docs/reference/kubernetes-api/workload-resources/*
III. *https://www.mirantis.com/blog/introduction-to-yaml-creating-a-kubernetes-deployment/*
IV. *https://spacelift.io/blog/kubernetes-deployment-yaml#how-do-i-create-a-yaml-file-for-kubernetes-deployment*

- Create a deployment file with the name `firstdeployment.yaml`
    - The first statement starts with *apiVersion*: this indicates the version of the Kubernetes API you're using to create this object.
        - Add `apiVersion: v1` into the file
    - Next, let's specify what kind of object we want to configure/create. For example Pod or Deployment.

- Add `kind: Pod` line

    - Next, we will define the metadata block. It helps to uniquely identify the object, including a name string, label, UID, and optionally the name of the namespace.
        - Name should be unique; a label can match a set of pods with the same functionality/task/goal (for example, all pods of the same backend API implementation).
        - Let's assign `first` as the name of the pod and `web` as the label. The code is shown below
        - Finally, mention the specification of the object itself, the Pod
            - Container image to use in the deployment. Let's use nginx web server container image (Automatically downloaded from Docker Hub by default)
            - We will also specify the name for the container and the port of the container to be exposed.
- The complete deployment file looks similar to the following code (*Be careful with copying these blocks. It requires spaces instead of tabs*):

```
---
apiVersion: v1
kind: Pod
metadata:
  name: first
  labels:
   app: web
spec:
  containers:
  - name: mynginx
      image: nginx
      ports:
      - containerPort: 80
```

- Run the YAML file: `kubectl create -f firstdeployment.yaml`
- Run the YAML file after modifications if any: `k3s kubectl apply -f firstdeployment.yaml`

- List the pods/ see that the pod is created: `k3s kubectl get pods -o wide`
- Get the additional info about pod: `k3s kubectl describe pod first`
    - You can go through the tail end output of the command, where it shows the list of activities that happened to spawn a pod.
- Get the VM or node IP of the nginx pod running `k3s kubectl describe pod first | grep IP`

Using the command from the last step, get an IP of pods. Curl it from inside the VM terminal. You should be able to see the webpage: curl <IP-address-of-a-pod> . Here IP address may look like 10.xx.xx.xx and output should look like below.

```
osboxes@osboxes:~/satish/CC/kube/ex8kube$ kubectl describe pod first | grep IP
IP:               10.42.0.53
IPs:
  IP:  10.42.0.53
osboxes@osboxes:~/satish/CC/kube/ex8kube$ curl 10.42.0.53
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, the nginx web server is successfully installed and
working. Further configuration is required.</p>

<p>For online documentation and support please refer to
<a href="http://nginx.org/">nginx.org</a>.<br/>
Commercial support is available at
<a href="http://nginx.com/">nginx.com</a>.</p>
```

Try accessing the nginx pod in your browser (http://CONTROLLER_VM_EXTERNAL_IP:80)



NB! At the end, delete the pod.

6. Message-Board flask application with Kubernetes

The main goal is to create a single pod instance using Kubernetes which will have the containers for Flask application and PostgreSQL relational database container to store the data.

Unlike docker-compose.yml file which has the provision to build docker images by accessing the Dockerfile within the folder via the 'build .' command, the Kubernetes YAML file cannot directly build a Docker image from the folder it resides in as it is designed to manage containerized applications. Instead, Docker images need to be pre-built and either stored locally or pushed to a container registry (e.g., Docker Hub) for Kubernetes to use.

So, first build your flask image. You built it already in task 8.1 of the exercise.

Tag your image before pushing into docker hub as : `<your-dockerhub-id>/flask2.0`

Push the image to docker hub.

NB! Show the image on the docker hub. Also show the result of your command. (1 point)

For the Flask application with PostgreSQL Database, let us create a .yaml file called flask_postgres.yaml in the lab2app folder or any other folder of your choice.

Configure the file (template is provided below) with your own postgres password and use the docker image which was pushed to Docker Hub for the flask application. You also have to fill the parts left out as part of your exercise.

Compared to docker-compose which is mainly suitable for local development and requires manual scaling, Kubernetes allows multiple replicas, has support for load-balancing, persistent storage, self-healing orchestration.

- *A Persistent Volume (PV) is the actual physical storage resource in the cluster, which represents the actual disk space on the node where the data is stored.*
- *A PersistentVolumeClaim (PVC) is a request for storage made by a pod. We can attach PVs to pods via PVC. Here, it is created to store the database data persistently. The Flask app connects to PostgreSQL using postgres (service name) and the PostgreSQL data persists even if the pod restarts due to PVC.*

Explanation of the flask_postgres.yaml file:

First, PVC is configured with ReadWriteOnce access mode (the volume can be mounted as read/ write by a single node) and it requests a PV for 1GiB of storage. This storage will be mounted inside the PostgreSQL container to persist data. Then, we define the pod with name as flask-postgres-pod which has two containers named flask-app and postgres. The postgres container runs the postgres:14.1-alpine image (by pulling it from Docker Hub) and it has the environment configuration (username, password) along with where the path for where the PVC volume should be mounted, which ensures persistence of the data and the port 5432 is exposed.

The Flask container runs the docker image which was built (<your-dockerhub-id>/flask2.0). It also connects to the PostgreSQL container using DATABASE_URL. Flask listens on 0.0.0.0 to accept external requests. The Pod requests storage from the postgres-pv and k3s will dynamically provision PV if it is not set up. In the flask-service Service, it exposes flask application to the network, maps internal port 80 to external port 5000 with the TCP protocol. It also uses NodePort exposes the service externally on a random, valid high port number (30000 to 32767).

Content of flask_postgres.yaml file:

```
---
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: postgres-pvc
spec:
  accessModes:
  - ReadWriteOnce
  resources:
    requests:
      storage: 1Gi
---
apiVersion: v1
kind: Pod
metadata:
  name: flask-postgres-pod
  labels:
            app: flask-app
spec:
  containers:
  - name: postgres
            image: postgres:14.1-alpine
            env:
            <Set the environment list for POSTGRES_USER, POSTGRES_PASSWORD and POSTGRES_DB>
            volumeMounts:
            - mountPath: /var/lib/postgresql/data
            name: postgres-storage
            ports:
            - containerPort: 5432

  - name: flask-app
            image: <Put your docker image from docker hub>
            env:
            - name: DATABASE_URL
            value: "postgresql://postgres:srirama@localhost:5432/postgres"
            - name: FLASK_RUN_HOST
            value: "0.0.0.0"
            ports:
            - containerPort: 5000

  volumes:
  - name: postgres-storage
            persistentVolumeClaim:
              claimName: postgres-pvc
```

```
---
apiVersion: v1
kind: Service
metadata:
  name: flask-service
spec:
  selector:
        app: flask-app
  ports:
  - protocol: TCP
        port: 80
        targetPort: 5000
  type: NodePort
  externalTrafficPolicy: Local
```

TODO (Exercise)**:** (1 point)

1. Fill the environment list for POSTGRES_USER, POSTGRES_PASSWORD and POSTGRES_DB in the flask_postgres.yaml
2. Fill the docker image you created for the flask application

- *To apply the YAML file deployment (in the default namespace)*

  `kubectl apply -f flask_postgres.yaml`

- *Verify whether the deployments are successful:*

  `kubectl get pods` or `kubectl get pods -o wide` (for a detailed view of the pod in the default namespace) Wait till the status of the pods changes from Pending -> ContainerCreating -> Running state

  1. Pending: pod is accepted by the Kubernetes scheduler but it is waiting for resources (for image to get pulled, PV is not bound to PVC yet, or no available nodes, etc.)
  2. ContainerCreating: The pod is scheduled and Kubernetes is preparing to start the containers.
  3. Running: All the containers in the pod are executing successfully without any errors.
  4. CrashLoopBackOff: Container(s) is failing and restarting repeatedly. The logs can be checked for more details.
  5. ImagePullBackOff: Kubernetes is unable to download the container image from the registry.
- *To check the services:*

  `kubectl get svc` or `kubectl get services`
- Check the logs to ensure everything is working correctly:

  `kubectl logs flask-postgres-pod -c flask-app` (flask-app)

  `kubectl logs flask-postgres-pod -c postgres` (postgres)
- *With kubectl get svc command, we get to know the external port which is exposed via NodePort to access the Flask application via the node's ip address. In this case, the port for external access if 32155 (which maps to the internal port 80 via TCP protocol).*
- *To check the Persistent Volume which was created:*

  `kubectl get pv` (It is using the local storage path)
- *To check the Persistent Volume Claim (PVC) created for this deployment:*

`kubectl get pvc` (postgres-pvc is Bound, meaning it successfully found a matching PV)

- *by using* `kubectl logs flask-postgres-pod -c flask-app` *get the IP address of the flask-app container*

```
osboxes@osboxes:~/satish/CC/kube/ex8kube$ kubectl get pods
NAME                READY   STATUS    RESTARTS   AGE
first               1/1     Running   0          80m
flask-postgres-pod  2/2     Running   0          76s
osboxes@osboxes:~/satish/CC/kube/ex8kube$ kubectl logs flask-postgres-pod -c fla
sk-app
 * Serving Flask app 'app.py' (lazy loading)
 * Environment: production
   WARNING: This is a development server. Do not use it in a production deployme
nt.
   Use a production WSGI server instead.
 * Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment.
 Use a production WSGI server instead.
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:5000
 * Running on http://10.42.0.55:5000
Press CTRL+C to quit
```

- *The flask-application is accessible at http://10.42.0.55:5000 from the same node and it is accessible via http://<node_ip>:32155 from any node in this network.*
- You can see the port mapped to by using the following command.
  `kubectl get svc`

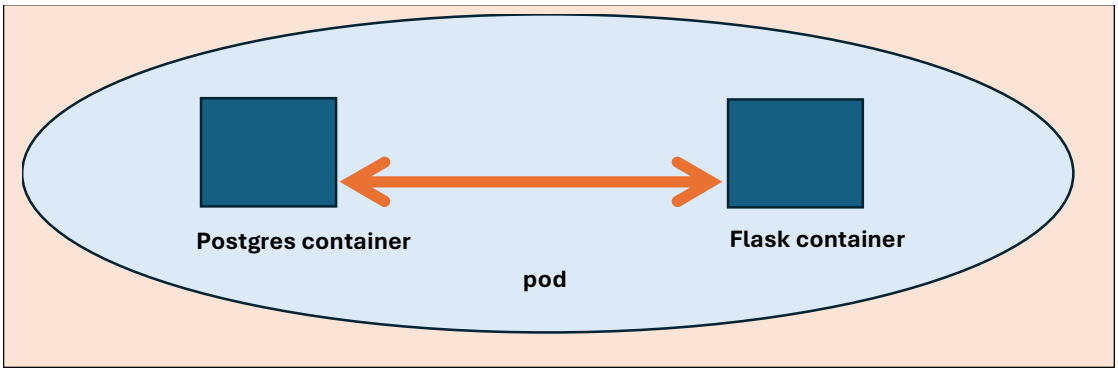
```
osboxes@osboxes:~/satish/CC/kube/ex8kube$ kubectl get svc
NAME            TYPE        CLUSTER-IP    EXTERNAL-IP   PORT(S)        AGE
flask-service   NodePort    10.43.9.78    <none>        80:32155/TCP   33m
kubernetes      ClusterIP   10.43.0.1     <none>        443/TCP        13d
```

- *Enter some text to verify whether the messages are getting displayed correctly*



- NB! Take the screenshot of your flask messages page (1 point)

The current setup of the containers in the pod is as follows. In this case, flask application container connects to PostgreSQL database using localhost:5432 since both containers can share the same network namespace inside the pod.



- *To check the messages are correctly stored in the PostgreSQL container:*
  `kubectl exec -it flask-postgres-pod -c postgres -- sh` (Opens a shell inside the PostgreSQL container)
  / # `psql -U postgres -d postgres` (Login with username as postgres, and connect to postgres database)
  postgres=# `\dt` (Display all available tables)
  `postgres=# SELECT * FROM messages;` (Retrieve content of the messages table)
  `postgres=# SELECT COUNT(*) FROM messages;` (Count total number of entries in the table)

```
osboxes@osboxes:~/satish/CC/kube/ex8kube$ kubectl exec -it flask-postgres-pod -c
 postgres -- sh
/ # psql -U postgres -d postgres
psql (14.1)
Type "help" for help.

postgres=# \dt
         List of relations
 Schema |   Name   | Type  |  Owner
--------+----------+-------+----------
 public | messages | table | postgres
(1 row)

postgres=# select * from messages;
 id |           message_text           |         timestamp
----+----------------------------------+----------------------------
  1 | Hi                               | 2025-03-22 18:13:52.685966
  2 | Hello                            | 2025-03-22 18:13:59.932739
  3 | Successfully finished exercise 8 | 2025-03-22 18:14:19.478679
(3 rows)

postgres=#
```

  - NB! Take the screenshot of your messages in the db. (1 point)
- NB! At the end, delete the pods.
- Now demonstrate the persistence is working as expected when you restart the pods. (1 point)

  NB! Show the commands and the screenshots

Observations

1. In the above configuration of 8.6, the system will work even when you enter wrong password for the DB. It is working so, as you are connecting to the pod from the same machine using localhost.

   This is due to the following reasons:

   Client authentication is controlled by a configuration file, which traditionally is named pg_hba.conf and is stored in the database cluster's data directory. (HBA stands for host-based authentication.)

   A default pg_hba.conf file is installed when the data directory is initialized by initdb. (Reference: https://www.postgresql.org/docs/current/auth-pg-hba-conf.html)

1. *local all all trust*

This applies to Unix domain socket connections (when connecting via localhost or /var/run/postgresql).
Effect: No password is required when connecting via a Unix socket.
This explains why the Flask app was able to connect even if the password was incorrect when using localhost.

2. *host all all 127.0.0.1/32 trust*

This applies to IPv4 connections from localhost (127.0.0.1).
Effect: No password is required for any user connecting from 127.0.0.1.
This means if Flask connects via 127.0.0.1, it does not need a password.

3. *host all all all scram-sha-256*

This rule applies to all other network connections.
Effect: Any connection not from localhost must use scram-sha-256 authentication (i.e., requires the correct password).

Hence, to avoid this, we need to replace all trust instances with scram-sha-256

To do this, first exec into the pod:

1. `kubectl exec -it flask-postgres-pod -- sh`
2. `su postgres` (switch to postgres user)

   `/ # cat /var/lib/postgresql/data/pg_hba.conf`

   `/ # vi /var/lib/postgresql/data/pg_hba.conf` (to open the file)

   Then, change all instances of trust to scram-sha-256

   In vi, ‘i' to insert text, ‘esc’ after done, ‘:wq’ (to save and exit)

3. After saving the changes, reload the new configuration via `pg_ctl reload`

Alternatively, you can achieve this by copying the correct pg_hba.conf file to the container, while preparing the .yaml file. You can use `exec` option with `args`. Try it out and provide the updated flask_postgres.yaml along with the detailed demonstration. (1 bonus point)

Follow these guidelines while submitting the exercises

1. Put a pdf file with your answers and screenshots for the questions and name it as your studentid_QE8
2. The screenshots are meant to show that the work is done and by you. So, have enough proofs in your answer sheet.
3. **Note**: Do not blindly copy the commands. Try to understand them. The provided commands are to guide you with the initial process.
4. Do not copy code from internet. If you copy, please leave a comment with the URL on that code/answer sheet
5. Plagiarism will not be tolerated and dealt through the formal channels of the university